EDS ASSIGNMENT 2

Applying NUMPY and PANDA methods to find the solution for the formulated problem statements.

Dataset: Enron Email

Name: Samruddhi Bhausaheb Mate

PRN: 202501040361

Batch: CS41

Loading the Data Set:

```python
import pandas as pd
import numpy as np

# ---------------------------------------------------------------------------
# Load Dataset
# ---------------------------------------------------------------------------
# header=5 skips the title/subtitle/stats/spacer/column-header rows
# row_index 0 in the sheet is the sub-header (descriptions), so we skip 1 more
df_raw = pd.read_excel("Enron_Email_Dataset_Enhanced_new.xlsx", header=4)

# Drop the first row which contains column description labels (e.g. "Total Words")
df = df_raw.iloc[1:].reset_index(drop=True)

# Rename columns to clean snake_case names
df.columns = [
    "Email_ID", "Date", "Day_of_Week", "Time_Sent", "Sender", "Recipient",
    "Subject", "Body_Preview", "Folder", "Category", "Word_Count",
    "Has_Attachment", "Priority", "Is_Forwarded", "Response_Time_Hrs",
    "Sentiment_Score", "Thread_Length", "Email_Size_KB", "Read_Receipt"
]

# Convert numeric columns to proper types
numeric_cols = ["Word_Count", "Response_Time_Hrs", "Sentiment_Score",
                "Thread_Length", "Email_Size_KB"]
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

print("Dataset loaded successfully.")
print(f"Shape: {df.shape[0]} rows x {df.shape[1]} columns\n")
print("=" * 65)
```

Data Set Loaded Successfully:

```
Dataset loaded successfully.
Shape: 100 rows x 19 columns
```

PANDAS OPERATIONS (1–10)

Pandas 1 — df.head() / df.info()

1. Problem Statement: The Enron email dataset has 19 columns of mixed types. We need to confirm column names, data types, and null counts are correct before performing any analysis

```
print("\n— PANDAS 1: Preview & Info ————————————————————————————")
print(df.head(5).to_string())
print()
df.info()
```

OUTPUT:

```
— PANDAS 1: Preview & Info ————————————————————
  Email_ID       Date Day_of_Week Time_Sent                Sender              Recipient                      Subject                                                                Body_Preview       Folder
as_Attachment Priority Is_Forwarded  Response_Time_Hrs  Sentiment_Score  Thread_Length  Email_Size_KB Read_Receipt
0  ENR-0001  1999-03-15     Monday    08:15  louise.kitchen@enron.com  trading.team@enron.com  Power Market Analysis – California  The California energy trading desk has exceeded daily limits. Action required.        Inbox
        Yes   Medium        Yes           27.75            0.22            2          33.7         No
1  ENR-0002  1999-03-25   Thursday    00:43   jeff.skilling@enron.com      managers@enron.com        401(k) Plan Changes Notice  The California energy trading desk has exceeded daily limits. Action required.      Flagged  C
         No      Low         No           69.19            0.62            3         488.5         No
2  ENR-0003  1999-04-04     Sunday    01:27  sherron.watkins@enron.com  all.employees@enron.com            SPE Disclosure Review  All personnel must complete online compliance training by end of this quarter.       Drafts  C
        Yes     High         No            1.85            0.80            1          89.0         No
3  ENR-0004  1999-04-14  Wednesday    14:23  sherron.watkins@enron.com           hr@enron.com       Project Condor Status Review  Legal has completed the SPE disclosure review. Three items flagged for action.  Deleted Items  C
         No   Medium        Yes           20.97           -0.47            5         657.2         No
4  ENR-0005  1999-04-24   Saturday    02:54   john.lavorato@enron.com        legal@enron.com          Azurix Asset Sale Update        FY2001 budget has been revised downward by 12%. Updated model attached.  Deleted Items
        Yes   Medium         No            1.73           -0.38            1         616.9        Yes

<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 19 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Email_ID         100 non-null    str
 1   Date             100 non-null    str
 2   Day_of_Week      100 non-null    str
 3   Time_Sent        100 non-null    str
 4   Sender           100 non-null    str
 5   Recipient        100 non-null    str
 6   Subject          100 non-null    str
 7   Body_Preview     100 non-null    str
 8   Folder           100 non-null    str
 9   Category         100 non-null    str
 10  Word_Count       100 non-null    int64
 11  Has_Attachment   100 non-null    str
 12  Priority         100 non-null    str
 13  Is_Forwarded     100 non-null    str
 14  Response_Time_Hrs  100 non-null  float64
 15  Sentiment_Score  100 non-null    float64
 16  Thread_Length    100 non-null    int64
 17  Email_Size_KB    100 non-null    float64
 18  Read_Receipt     100 non-null    str
dtypes: float64(3), int64(2), str(14)
memory usage: 15.0 KB
```

Pandas 2 — df.describe()

2. Problem Statement: We need to understand the statistical range of numeric columns like Word_Count, Sentiment_Score, and Response_Time_Hrs. This helps detect outliers and verify data quality**.**

```
print("\n— PANDAS 2: Descriptive Statistics ————————————————————")
stats = df.describe()
print(stats.to_string())
```

OUTPUT:

```
— PANDAS 2: Descriptive Statistics ————————————————————
        Word_Count  Response_Time_Hrs  Sentiment_Score  Thread_Length  Email_Size_KB
count   100.000000         100.000000       100.000000      100.00000     100.000000
mean    775.710000          16.047200        -0.024800        3.28000     416.934000
std     361.341001          15.383939         0.580265        2.82514     247.726586
min     137.000000           0.530000        -0.900000        1.00000      14.100000
25%     486.750000           4.032500        -0.600000        1.00000     196.600000
50%     718.500000          11.145000         0.220000        2.00000     430.250000
75%    1057.750000          23.470000         0.580000        4.00000     619.850000
max    1542.000000          69.190000         0.800000       14.00000     840.200000
```

Pandas 3 — df[df[col] == value]

3. Problem statement: Out of 100 emails, we need to isolate only HIGH priority messages for an urgent audit. Manually scanning all rows is not feasible, so we apply a condition-based row filter on the Priority column

```
print("\n— PANDAS 3: Filter – High Priority Emails ——————————")
high_priority = df[df["Priority"] == "High"]
print(f"Total High Priority emails: {len(high_priority)}")
print(high_priority[["Email_ID", "Sender", "Subject", "Priority"]].head(8).to_string())
```

OUTPUT:

```
── PANDAS 3: Filter — High Priority Emails ──────────────
Total High Priority emails: 43
    Email_ID                    Sender                                Subject Priority
2   ENR-0003  sherron.watkins@enron.com                SPE Disclosure Review     High
8   ENR-0009    andrew.fastow@enron.com          Project Condor Status Review     High
10  ENR-0011   vince.kaminski@enron.com  Mandatory Compliance Training Notice     High
12  ENR-0013       rex.shelby@enron.com              Azurix Asset Sale Update     High
13  ENR-0014    andrew.fastow@enron.com        Arthur Andersen Audit Findings     High
17  ENR-0018   louise.kitchen@enron.com        New VP Operations Announcement     High
18  ENR-0019   richard.causey@enron.com        New VP Operations Announcement     High
19  ENR-0020  sara.shackleton@enron.com              Wholesale Trading Limits     High
```

Pandas 4 — df.groupby().mean()

4. Problem Statement: We want to compare email categories (Legal, Finance, HR, etc.) to find which ones have the longest emails and slowest responses. Grouping by Category and averaging the metrics reveals this pattern

```
print("\n── PANDAS 4: GroupBy Category — Avg Word Count & Response Time —")
grouped = df.groupby("Category")[["Word_Count", "Response_Time_Hrs"]].mean().round(2)
grouped.columns = ["Avg_Word_Count", "Avg_Response_Time_Hrs"]
print(grouped.sort_values("Avg_Response_Time_Hrs", ascending=False).to_string())
```

OUTPUT:

```
── PANDAS 4: GroupBy Category — Avg Word Count & Response Time —
            Avg_Word_Count  Avg_Response_Time_Hrs
Category
Operations          843.00                  23.08
Trading             784.56                  20.00
Risk                732.36                  18.77
Executive           866.86                  17.24
HR                  687.43                  16.39
Audit               711.85                  13.24
Financial           723.22                  11.40
Compliance          878.33                   8.73
Legal               730.17                   7.99
```

Pandas 5 — df.sort_values()

5. Problem Statement: Investigators need to review the most negatively toned emails first to identify hostile or suspicious communications in the dataset. Sorting by Sentiment_Score ascending brings the worst emails to top.

```
print("\n── PANDAS 5: Sort by Sentiment Score (Most Negative First) —")
sorted_df = df.sort_values("Sentiment_Score", ascending=True)
print(sorted_df[["Email_ID", "Sender", "Subject", "Sentiment_Score"]].head(8).to_string())
```

OUTPUT:

```
── PANDAS 5: Sort by Sentiment Score (Most Negative First) ─
    Email_ID                  Sender                           Subject  Sentiment_Score
25  ENR-0026     tana.jones@enron.com            JEDI Partnership Review            -0.90
28  ENR-0029   beth.prentice@enron.com     Q4 Performance Review Schedule            -0.90
80  ENR-0081  louise.kitchen@enron.com           LJM Contract Final Terms            -0.90
75  ENR-0076  louise.kitchen@enron.com       Project Condor Status Review            -0.90
86  ENR-0087  louise.kitchen@enron.com  Power Market Analysis – California            -0.90
24  ENR-0025     tana.jones@enron.com         SEC Inquiry Response Draft            -0.81
7   ENR-0008    greg.whalley@enron.com   Energy Trading Update: CA Markets            -0.81
13  ENR-0014   andrew.fastow@enron.com     Arthur Andersen Audit Findings            -0.81
```

Pandas 6 — df.isnull() / df.fillna()

6.Problem Statement: Missing values in numeric columns like Response Time_Hrs and Sentiment_Score will cause errors in calculations and ML models. We detect all nulls and fill them with the column median value.

```
print("\n── PANDAS 6: Missing Values Detection & Fill ────────────────")
null_counts = df.isnull().sum()
print("Null counts per column:")
print(null_counts[null_counts >= 0].to_string())

for col in numeric_cols:
    median_val = df[col].median()
    df[col] = df[col].fillna(median_val)

print(f"\nAfter fillna – remaining nulls in numeric cols: "
      f"{df[numeric_cols].isnull().sum().sum()}")
```

OUTPUT:

```
── PANDAS 6: Missing Values Detection & Fill ────────────────
Null counts per column:
Email_ID             0
Date                 0
Day_of_Week          0
Time_Sent            0
Sender               0
Recipient            0
Subject              0
Body_Preview         0
Folder               0
Category             0
Word_Count           0
Has_Attachment       0
Priority             0
Is_Forwarded         0
Response_Time_Hrs    0
Sentiment_Score      0
Thread_Length        0
Email_Size_KB        0
Read_Receipt         0

After fillna – remaining nulls in numeric cols: 0
```

Pandas 7 — df["new_col"] = expression

7. Problem Statement: Email_Size_KB alone does not tell us if a message is text-heavy or attachment-heavy. We create a new Words_Per_KB column by dividing Word_Count by Email_Size_KB to measure text density

```
print("\n── PANDAS 7: Add Computed Column — Words_Per_KB ──────────────")
df["Words_Per_KB"] = (df["Word_Count"] / df["Email_Size_KB"]).round(3)
print(df[["Email_ID", "Word_Count", "Email_Size_KB", "Words_Per_KB"]].head(10).to_string())
```

OUTPUT:

```
── PANDAS 7: Add Computed Column — Words_Per_KB ──────────
   Email_ID  Word_Count  Email_Size_KB  Words_Per_KB
0  ENR-0001         268           33.7         7.953
1  ENR-0002         969          488.5         1.984
2  ENR-0003         684           89.0         7.685
3  ENR-0004         683          657.2         1.039
4  ENR-0005         557          616.9         0.903
5  ENR-0006        1325          646.6         2.049
6  ENR-0007        1385          710.1         1.950
7  ENR-0008        1447           21.1        68.578
8  ENR-0009        1323          331.6         3.990
9  ENR-0010        1011          597.5         1.692
```

Pandas 8 — df[col].value_counts()

8. Problem Statement: We need to identify which employees sent the most emails in this dataset. Unusually high sender frequency can indicate key personnel or suspicious activity worth flagging for further investigation.

```
print("\n── PANDAS 8: Value Counts — Emails per Sender ──────────────")
sender_counts = df["Sender"].value_counts()
print(sender_counts.to_string())
```

OUTPUT:

```
── PANDAS 8: Value Counts — Emails per Sender ──────────────
Sender
tana.jones@enron.com          13
louise.kitchen@enron.com      11
vince.kaminski@enron.com      10
sherron.watkins@enron.com      9
john.lavorato@enron.com        9
sara.shackleton@enron.com      8
ken.lay@enron.com              7
jeff.skilling@enron.com        6
beth.prentice@enron.com        6
andrew.fastow@enron.com        5
richard.causey@enron.com       4
kevin.hannon@enron.com         4
greg.whalley@enron.com         3
rex.shelby@enron.com           3
mark.taylor@enron.com          2
```

Pandas 9 — df.groupby().agg()

9 . Problem Statement : A single metric per priority level (Low/Normal/High) is not enough. We need a combined view of email count, average sentiment, average response time, and total thread length — all grouped by Priority.

```
print("\n— PANDAS 9: Multi-Aggregation by Priority ——————————————")
priority_agg = df.groupby("Priority").agg(
    Email_Count       =("Email_ID",          "count"),
    Avg_Sentiment     =("Sentiment_Score",    "mean"),
    Avg_Response_Hrs  =("Response_Time_Hrs",  "mean"),
    Total_Thread_Len  =("Thread_Length",      "sum")
).round(3)
print(priority_agg.to_string())
```

OUTPUT:

```
— PANDAS 9: Multi-Aggregation by Priority ——————————————
          Email_Count  Avg_Sentiment  Avg_Response_Hrs  Total_Thread_Len
Priority
High               43         -0.078            15.646               144
Low                19         -0.091            15.043                65
Medium             38          0.069            17.004               119
```

Pandas 10 — df.corr()

10 .Problem Statement : Before building any predictive model, we must know which numeric features are linearly related to each other. For example, does a higher Word_Count lead to longer Response_Time_Hrs or Thread_Length?

```
print("\n— PANDAS 10: Correlation Matrix (Numeric Columns) ——————————")
corr_matrix = df[numeric_cols + ["Words_Per_KB"]].corr(numeric_only=True).round(3)
print(corr_matrix.to_string())
```

OUTPUT:

```
— PANDAS 10: Correlation Matrix (Numeric Columns) ————————
                 Word_Count  Response_Time_Hrs  Sentiment_Score  Thread_Length  Email_Size_KB  Words_Per_KB
Word_Count            1.000              0.127           -0.074          0.061         -0.026         0.277
Response_Time_Hrs     0.127              1.000           -0.007         -0.016          0.019         0.175
Sentiment_Score      -0.074             -0.007            1.000         -0.126         -0.103         0.004
Thread_Length         0.061             -0.016           -0.126          1.000          0.023        -0.124
Email_Size_KB        -0.026              0.019           -0.103          0.023          1.000        -0.519
Words_Per_KB          0.277              0.175            0.004         -0.124         -0.519         1.000
```

NUMPY OPERATIONS (1–10)

NumPy 1 — np.mean() / np.median()

Problem Statement: We need the central tendency of Word_Count, Sentiment_Score, and Response_Time_Hrs to understand typical email behaviour. Comparing mean vs median tells us if the data is skewed by outliers

```
print("\n\n— NUMPY 1: Mean & Median ———————————————————————————")
print(f"Word Count  – Mean: {np.mean(word_arr):.2f},  Median: {np.median(word_arr):.2f}")
print(f"Sentiment   – Mean: {np.mean(sent_arr):.3f},  Median: {np.median(sent_arr):.3f}")
print(f"Response Hr – Mean: {np.mean(resp_arr):.2f},  Median: {np.median(resp_arr):.2f}")
```

OUTPUT:

```
— NUMPY 1: Mean & Median ———————————————————————————
Word Count  – Mean: 775.71,  Median: 718.50
Sentiment   – Mean: -0.025,  Median: 0.220
Response Hr – Mean: 16.05,  Median: 11.14
```

NumPy 2 — np.std() / np.var()

Problem Statement: We need to measure how consistent email response times and sizes are. A high standard deviation in Response_Time_Hrs signals an unreliable communication system with unpredictable employee behaviour.

```
print("\n— NUMPY 2: Standard Deviation & Variance ——————————————")
print(f"Response Time – Std Dev: {np.std(resp_arr):.2f} hrs,  Variance: {np.var(resp_arr):.2f}")
print(f"Email Size KB – Std Dev: {np.std(size_arr):.2f} KB,   Variance: {np.var(size_arr):.2f}")
```

OUTPUT:

```
— NUMPY 2: Standard Deviation & Variance ——————————————
Response Time – Std Dev: 15.31 hrs,  Variance: 234.30
Email Size KB – Std Dev: 246.48 KB,   Variance: 60754.78
```

NumPy 3 — np.min() / np.max() / np.ptp()

Problem Statement: We need the exact data boundaries for Word_Count, Sentiment_Score, Response_Time_Hrs, and Email_Size_KB before applying any scaling. The range (max - min) also reveals how spread out each feature is.

```
print("\n— NUMPY 3: Min, Max & Range ———————————————————————————")
for name, arr in [("Word Count", word_arr), ("Sentiment", sent_arr),
                  ("Response Hrs", resp_arr), ("Email Size KB", size_arr)]:
    print(f"{name:15s} – Min: {np.min(arr):>8.2f}  "
          f"Max: {np.max(arr):>8.2f}  Range: {np.ptp(arr):>8.2f}")
```

OUTPUT:

```
— NUMPY 3: Min, Max & Range ———————————————————————————
Word Count      – Min:   137.00  Max:  1542.00  Range:  1405.00
Sentiment       – Min:    -0.90  Max:     0.80  Range:     1.70
Response Hrs    – Min:     0.53  Max:    69.19  Range:    68.66
Email Size KB   – Min:    14.10  Max:   840.20  Range:   826.10
```

NumPy 4 — np.percentile()

Problem Statement: Mean and std can mislead when data is skewed or has outliers. We compute the 25th, 50th, 75th, and 90th percentiles of Word_Count and Response_Time_Hrs to get a robust picture of their distribution.

```
print("\n— NUMPY 4: Percentiles (25 / 50 / 75 / 90) ——————————")
for name, arr in [("Word Count", word_arr), ("Response Hrs", resp_arr)]:
    p = np.percentile(arr, [25, 50, 75, 90])
    print(f"{name:15s} – P25: {p[0]:.1f}  P50: {p[1]:.1f}  P75: {p[2]:.1f}  P90: {p[3]:.1f}")
```

OUTPUT:

```
— NUMPY 4: Percentiles (25 / 50 / 75 / 90) ——————————
Word Count      – P25: 486.8  P50: 718.5  P75: 1057.8  P90: 1323.1
Response Hrs    – P25: 4.0  P50: 11.1  P75: 23.5  P90: 40.2
```

NumPy 5 — np.reshape()

Problem Statement: Machine learning models and matrix operations require input data in 2D format, not a flat 1D array. We reshape the 100 Word_Count values from shape (100,) into a 10×10 matrix for batch processing.

```
print("\n— NUMPY 5: Array Reshape – Word Count into 10×10 Matrix ——")
word_matrix = word_arr[:100].reshape(10, 10)
print(f"Original shape: {word_arr[:100].shape}  →  Reshaped: {word_matrix.shape}")
print("First 3 rows of reshaped matrix:")
print(word_matrix[:3])
```

OUTPUT:

```
— NUMPY 5: Array Reshape – Word Count into 10×10 Matrix ——
Original shape: (100,)  →  Reshaped: (10, 10)
First 3 rows of reshaped matrix:
[[ 268.  969.  684.  683.  557. 1325. 1385. 1447. 1323. 1011.]
 [ 391.  658. 1040.  456.  875.  963.  370.  603.  591.  948.]
 [1466.  487. 1504.  652.  486.  667.  501.  592. 1087. 1056.]]
```

NumPy 6 — Boolean Masking

Problem Statement: We need to automatically flag emails with a Sentiment_Score below -0.5, which indicates strongly negative or hostile language. Boolean masking filters these records instantly without any loop.

```
print("\n— NUMPY 6: Boolean Masking – Negative Sentiment Emails ———")
negative_mask   = sent_arr < -0.5
negative_scores = sent_arr[negative_mask]
negative_ids    = df["Email_ID"].to_numpy()[negative_mask]
print(f"Emails with Sentiment < -0.5 : {negative_mask.sum()} emails")
print("Email IDs:", negative_ids[:10])
print("Scores:   ", np.round(negative_scores[:10], 3))
```

OUTPUT:

```
— NUMPY 6: Boolean Masking – Negative Sentiment Emails ——
Emails with Sentiment < -0.5 : 31 emails
Email IDs: ['ENR-0008' 'ENR-0009' 'ENR-0010' 'ENR-0013' 'ENR-0014' 'ENR-0017'
 'ENR-0019' 'ENR-0022' 'ENR-0023' 'ENR-0025']
Scores:    [-0.81 -0.6  -0.55 -0.6  -0.81 -0.55 -0.55 -0.6  -0.81 -0.81]
```

NumPy 7 — Min-Max Normalisation

Problem Statement: Word_Count and Email_Size_KB are on very different numeric scales, which causes bias in distance-based ML algorithms like KNN or SVMW e normalise both features to the [0, 1] range to fix this issue.

```
print("\n— NUMPY 7: Min-Max Normalisation (0-1 Scaling) ————————")
def minmax_norm(arr):
    return (arr - np.min(arr)) / (np.max(arr) - np.min(arr))

word_norm = minmax_norm(word_arr)
size_norm = minmax_norm(size_arr)
print(f"Word Count  normalised – Min: {word_norm.min():.3f}  Max: {word_norm.max():.3f}  Mean: {word_norm.mean():.3f}")
print(f"Email Size  normalised – Min: {size_norm.min():.3f}  Max: {size_norm.max():.3f}  Mean: {size_norm.mean():.3f}")
print("First 10 normalised Word Count values:", np.round(word_norm[:10], 4))
```

OUTPUT:

```
— NUMPY 7: Min-Max Normalisation (0-1 Scaling) ————————
Word Count  normalised – Min: 0.000  Max: 1.000  Mean: 0.455
Email Size  normalised – Min: 0.000  Max: 1.000  Mean: 0.488
First 10 normalised Word Count values: [0.0932 0.5922 0.3893 0.3886 0.2989 0.8456 0.8883 0.9324 0.8441 0.6221]
```

NumPy 8 — np.unique()

Problem Statement: We need to identify all distinct senders in the dataset and count how many emails each person sent, without using Pandas. This helps detect dominant senders using pure NumPy frequency analysis.

```
print("\n— NUMPY 8: Unique Senders & Email Counts ————————————————")
sender_arr            = df["Sender"].to_numpy()
unique_senders, counts = np.unique(sender_arr, return_counts=True)
sort_idx = np.argsort(counts)[::-1]
print(f"Unique senders found: {len(unique_senders)}")
print(f"\n{'Sender':<40} {'Count':>5}")
print("-" * 47)
for s, c in zip(unique_senders[sort_idx], counts[sort_idx]):
    print(f"{s:<40} {c:>5}")
```

OUTPUT:

```
── NUMPY 8: Unique Senders & Email Counts ──────────────────
Unique senders found: 15

Sender                                        Count
-----------------------------------------------------
tana.jones@enron.com                             13
louise.kitchen@enron.com                         11
vince.kaminski@enron.com                         10
john.lavorato@enron.com                           9
sherron.watkins@enron.com                         9
sara.shackleton@enron.com                         8
ken.lay@enron.com                                 7
beth.prentice@enron.com                           6
jeff.skilling@enron.com                           6
andrew.fastow@enron.com                           5
richard.causey@enron.com                          4
kevin.hannon@enron.com                            4
rex.shelby@enron.com                              3
greg.whalley@enron.com                            3
mark.taylor@enron.com                             2
```

NumPy 9 — np.cumsum()

Problem Statement: We want to track how total word volume grows across all 100 emails in sequence. The cumulative sum shows at which point communication load crossed key milestones like 5,000 or 10,000 total words.

```
print("\n── NUMPY 9: Cumulative Word Count (np.cumsum) ────────────────")
cum_words = np.cumsum(word_arr)
print(f"Total words across all emails: {int(cum_words[-1]):,}")
print(f"\nCumulative word count at every 10th email:")
for i in range(9, len(cum_words), 10):
    print(f"  Email #{i+1:>3} — Cumulative Words: {int(cum_words[i]):>7,}")
```

OUTPUT:

```
── NUMPY 9: Cumulative Word Count (np.cumsum) ──────────────
Total words across all emails: 77,571

Cumulative word count at every 10th email:
  Email # 10 — Cumulative Words:   9,652
  Email # 20 — Cumulative Words:  16,547
  Email # 30 — Cumulative Words:  25,045
  Email # 40 — Cumulative Words:  33,375
  Email # 50 — Cumulative Words:  41,193
  Email # 60 — Cumulative Words:  47,779
  Email # 70 — Cumulative Words:  54,525
  Email # 80 — Cumulative Words:  62,979
  Email # 90 — Cumulative Words:  70,369
  Email #100 — Cumulative Words:  77,571
```

NumPy 10 — np.histogram()

Problem Statement: We need to understand the full distribution of Sentiment_Score across all emails — not just the average.
Binning scores into 10 intervals reveals whether Enron emails lean positive, negative, or neutral.

```
print("\n── NUMPY 10: Histogram — Sentiment Score Distribution ────────")
bin_counts, bin_edges = np.histogram(sent_arr, bins=10)
print(f"{'Bin Range':<25} {'Count':>6}  {'Bar'}")
print("-" * 55)
for i in range(len(bin_counts)):
    lo  = bin_edges[i]
    hi  = bin_edges[i + 1]
    bar = "▇" * bin_counts[i]
    print(f"  [{lo:+.2f} → {hi:+.2f}]    {bin_counts[i]:>4}   {bar}")

print("\n" + "=" * 65)
print("All 10 Pandas + 10 NumPy operations completed successfully.")
print("=" * 65)
```

OUTPUT:

```
── NUMPY 10: Histogram — Sentiment Score Distribution ────────
Bin Range                  Count  Bar
-------------------------------------------------------
  [-0.90 → -0.73]      12   ▇▇▇▇▇▇▇▇▇▇▇▇
  [-0.73 → -0.56]      14   ▇▇▇▇▇▇▇▇▇▇▇▇▇▇
  [-0.56 → -0.39]       9   ▇▇▇▇▇▇▇▇▇
  [-0.39 → -0.22]      12   ▇▇▇▇▇▇▇▇▇▇▇▇
  [-0.22 → -0.05]       0
  [-0.05 → +0.12]       0
  [+0.12 → +0.29]      10   ▇▇▇▇▇▇▇▇▇▇
  [+0.29 → +0.46]      15   ▇▇▇▇▇▇▇▇▇▇▇▇▇
  [+0.46 → +0.63]      18   ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
  [+0.63 → +0.80]      10   ▇▇▇▇▇▇▇▇▇▇
```